A Stem Agent that Evolves into a Deep Research Agent

Josep Biel

1 Problem framing

The task asks for a *stem agent*: not a universal agent, and not a hand-built specialist, but a minimal agent that becomes specialized through its own process. I chose **technical deep research about LLM agents** as the task class. This domain is narrow enough to evaluate, but complex enough to require agentic behavior: question decomposition, search planning, source triage, evidence extraction, coverage checking, contradiction handling, citation auditing, and synthesis.

My interpretation of "becoming specific" is not unrestricted self-modification. Instead, the stem agent evolves an auditable **agent genome**: a YAML configuration describing workflow steps, prompt roles, allowed tools, memory policy, safeguards, trace requirements, and acceptance criteria. This keeps the transformation inspectable and safe. The code remains fixed; the behavior changes through a validated genome.

The core hypothesis is:

> A minimal research agent that only searches and summarizes should be improved by a constrained specialization process that makes deep-research subskills explicit and evaluates candidate genomes before accepting them.

The project therefore has two goals. First, the final agent should perform better than the baseline. Second, the repository should preserve the path by which the agent specialized, including failed intermediate versions.

2 Approach

2.1 Baselines

I implemented two baselines before building the evolved agent.

- **Model-only baseline**: answers from model knowledge, with no external tools. This isolates how much the model can do without retrieval.
- **Web-search baseline**: performs a simple retrieve-and-summarize pass using the OpenAI Responses API with web search. This isolates the value of retrieval without a specialized workflow.

The web baseline is intentionally under-specialized:

$$\text{question} \rightarrow \text{search/read top sources} \rightarrow \text{summarize with citations.}$$

It has no explicit source triage, evidence table, coverage audit, contradiction audit, citation audit, or run-scoped memory.

2.2 The genome

The evolved agent is represented by a genome validated against a project-level schema before execution. The schema requires specific sections and trace artifacts: workflow, prompt roles, tool boundaries, limits, memory policy, safeguards, output contract, evaluation metrics, acceptance criteria, and rollback policy.

The initial evolved workflow was:

$$\begin{aligned}\text{question} \rightarrow &\text{decompose} \rightarrow \text{search plan} \rightarrow \text{collect sources} \rightarrow \text{source triage} \\ \rightarrow &\text{extract evidence} \rightarrow \text{coverage check} \rightarrow \text{contradiction check} \\ \rightarrow &\text{synthesis} \rightarrow \text{citation audit} \rightarrow \text{limitations.}\end{aligned}$$

The tool boundary is deliberately narrow: the evolved agent still only uses web search. This is important because I wanted the comparison to test specialization of workflow, not simply "more tools". Arbitrary code execution, filesystem modification, source modification, and unbounded browsing are forbidden.

2.3 Safeguards

The stem-agent process includes several safeguards:

- **Schema validation**: evolved genomes must satisfy the genome contract before running.
- **Immutable boundaries**: the domain, evaluation set, rubric, and tool registry are fixed.

- **Trace contract**: each evolved run must produce artifacts for decomposition, search planning, source triage, evidence extraction, coverage audit, contradiction audit, citation audit, and final answer.
- **Budget limits**: each genome declares maximum search queries, sources, model calls, runtime, and evolution rounds.
- **Rollback criteria**: candidates are rejected if validation fails, smoke tests fail, or evaluation regresses.

The first implementation executes the evolved workflow in one model call while still requiring structured artifacts for each stage. This was a pragmatic compromise: it made the evolved architecture runnable and inspectable before adding a more expensive multi-call orchestrator.

3 Evaluation

3.1 Dataset and metrics

The fixed evaluation set contains eight technical research questions about LLM agents, with stable IDs DR-001 to DR-008. Each question includes required aspects and expected source types. Examples include memory failure modes, ReAct vs planner-executor agents, tool-use evaluation, agentic RAG, self-improving agents, multi-agent failure modes, citation auditing, and autonomous vs workflow-based agents.

I used two evaluation layers.

1. **Heuristic scorer**: deterministic and local. It checks coverage terms, citation presence, source domains, unsupported claim lines, uncertainty language, redundancy, runtime, and usage.
2. **Model-assisted judge**: stricter semantic evaluation using a fixed rubric. It grades factual accuracy, substantive coverage, evidence quality, uncertainty handling, usefulness for an engineer, and conciseness/structure. It also returns coverage, citation, and source audits.

The final score is:

$$\text{final} = 0.35 \cdot \text{heuristic} + 0.65 \cdot \text{judge}.$$

The judge is not treated as ground truth. It is a consistent semantic rubric, not an oracle. I report heuristic and judge scores separately because the first heuristic scorer overestimated polished but shallow answers. On one DR-001 baseline trace, the heuristic score was 0.9222 while the judge score was only 0.5417. This failure changed the evaluation design: the heuristic layer remains useful for diagnostics, but the model-assisted judge became only one part of the evidence.

For this reason, I do not claim that the agent evolved merely because a judge score increased. I claim evolution only when several signals agree: the same fixed questions are used, the same rubric is applied, deterministic trace metrics improve, concrete failure modes disappear, and the saved artifacts show a more specialized workflow. The judge score is useful for semantic pressure, but the strongest evidence is trace-level behavior such as source triage, evidence extraction, contradiction handling, citation audits, and removal of unsupported claim lines.

3.2 Evolution path

I kept five evolved genome variants as reproducible artifacts:

- **v1**: the agent infers required aspects from the question. Budget: 4 search queries, 6 sources, medium reasoning.
- **v2**: the runner injects the fixed evaluation requirements into the prompt so the agent must explicitly account for each aspect. Same search/source budget as v1.
- **v3**: same requirement injection as v2, but with lower reasoning effort and tighter budgets: 2 search queries and 4 sources.
- **v4**: starts again from v2 rather than v3, keeps the v2 budget, and adds explicit source-quality discipline: authority ranking, weak-source rejection, direct-support labeling, and inference labeling.
- **v5**: keeps v4's source-quality policy and adds a raw-URL citation contract, plus stricter filtering so rejected sources do not become final trace citations.

This path is important because it was not monotonic. v1 made the architecture observable but did not beat the web baseline. v2 substantially improved quality but was expensive. v3 reduced cost, but lost much of v2's quality. v4 improved quality again by targeting the judge's source-quality feedback, but increased cost and exposed a citation-format failure on one question. v5 fixed that failure and, in this run, improved both quality and cost. That path is one of the main lessons of the project.

Agent	Heuristic	Judge	Final	Combined tokens
Model-only baseline	0.3277	0.4167	0.3856	34,514
Web-search baseline	0.8417	0.5833	0.6738	158,887
Evolved v1	0.8133	0.5781	0.6604	306,817
Evolved v2	0.8935	0.7240	0.7833	380,138
Evolved v3	0.8164	0.6146	0.6852	193,890
Evolved v4	0.8862	0.7552	0.8010	450,061
Evolved v5	**0.9492**	**0.8073**	**0.8569**	354,487

Table 1: Aggregate live evaluation results over the fixed 8-question set.

4 Results

Table 1 shows the aggregate live results over all eight questions.

The model-only baseline performed poorly because it had no citations and no fresh evidence. Adding web search produced a large gain: final score improved from 0.3856 to 0.6738. This confirmed that retrieval is essential for deep research, but it also created a stronger baseline for the evolved agent. From that point onward, the question was not whether search helps; it was whether a stem agent can become a more specific research workflow than a simple retrieve-and-summarize baseline.

The best-quality genome was **v5**. Compared with the web-search baseline, v5 improved the judge score from 0.5833 to 0.8073 and the final score from 0.6738 to 0.8569. This is a final-score gain of +0.1831. The score is not the whole argument: v5 also produced the required structured artifacts, preserved accepted/rejected source decisions, improved citation-support diagnostics, and removed the major unsupported-claim failure seen in v4. Compared with v4, v5 improved final score by +0.0559 and judge score by +0.0521, while reducing combined tokens from 450,061 to 354,487.

The most important v5 result was DR-002. v4 had regressed there because the model used provider-style citation markers rather than literal URLs, so the heuristic scorer counted 12 unsupported claim lines. v5 raised DR-002 final score from 0.5741 to 0.8785, restored citation support from 0.0000 to 1.0000, and removed the provider-marker warning pattern.

v5 was still not perfect. It regressed against v4 on DR-001 and DR-008, though it stayed above the web baseline on both. DR-003 also produced a parse warning in the recorded live trace: the model returned almost-valid JSON, but the runner fell back to raw text instead of recovering structured artifacts. I treated this as a trace robustness issue rather than an answer-quality failure and later hardened the parser to recover valid leading JSON objects with harmless trailing text. A post-fix full rerun completed all 8 traces with no parse warnings and no citation-contract warnings; its judge average stayed at 0.8073 and final score moved only from 0.8569 to 0.8594, so I treat it as robustness validation rather than a new genome improvement.

v3 remains useful as a negative result. It used about 1.22x the web baseline's combined tokens and still slightly improved final score (0.6852 vs 0.6738), but its quality was unstable. It improved DR-004, DR-006, and DR-008, but regressed badly on DR-007, where the judge noted thin evidence and a low-quality Reddit citation. This suggests that aggressive budget reduction can remove too much evidence diversity for citation-sensitive research tasks.

Comparison vs web baseline	Judge delta	Final delta	Token multiplier
Evolved v1	-0.0052	-0.0134	1.93x
Evolved v2	**+0.1407**	**+0.1095**	2.39x
Evolved v3	+0.0313	+0.0114	1.22x
Evolved v4	+0.1719	+0.1272	2.83x
Evolved v5	**+0.2240**	**+0.1831**	2.23x

Table 2: Quality/cost trade-off of evolved variants relative to the web-search baseline.

5 What surprised me and what failed

The first evaluator was too optimistic. My initial heuristic scorer rewarded citation presence and reputable domains, so a shallow web-search answer looked excellent. The model-assisted judge exposed that the answer was directionally correct but missed substantive aspects. This was the most useful evaluation failure because it changed the project from optimizing surface form to evaluating semantic research quality.

Retrieval alone was a very strong baseline. The web-search baseline improved final score by +0.2882 over the model-only baseline. This meant the evolved agent had to beat not just an LLM, but a simple retrieval-augmented agent.

Specialization helped only when the task contract was explicit. v1 asked the agent to infer required aspects, but it did not beat the web baseline. v2 improved when the runner injected the fixed required aspects

from the evaluation question. This made coverage much more reliable, but it also exposes a limitation: the system benefits from having a structured task contract. In a future version, I would split the dataset into dev and held-out test sets and avoid injecting test-specific aspects into the final comparison.

Cost tuning was not free. v3 reduced combined tokens from 380,138 to 193,890, but its final score dropped from 0.7833 to 0.6852. The largest failure was DR-007, where source quality degraded. This suggests that citation-audit tasks need enough source diversity to avoid brittle or weak evidence.

Source-quality rules found a new systems issue. v4 improved the average judge score and raised the heuristic source-quality signal, but it exposed a citation-contract failure on DR-002. v5 fixed that by requiring raw URLs on claim lines and filtering final trace citations more carefully. The remaining issue was not the same: DR-003 showed that the runner needed more tolerant JSON recovery when the model returned an almost-valid object, so I added that recovery as a final robustness fix.

The workflow is still not a full multi-call agent. The current evolved runner executes a structured workflow in one model call. It is inspectable because it must return artifacts for every stage, but the stages are not independently executed or recoverable. A stronger version would split planning, retrieval, evidence extraction, auditing, and synthesis into separate calls with intermediate validation.

6 What I would do with more time

First, I would separate evolution-time and final evaluation more rigorously. The current project uses a fixed 8-question set and reports all results, but v2/v3 also use question-level required aspects as an explicit task contract. A better design would use a development set for genome selection and a held-out test set for the final before/after result.

Second, I would turn the evolved runner into a real multi-step workflow: planner call, search call, source triage call, evidence extraction call, coverage/citation audit calls, and final synthesis. This would make rollback and repair possible at the step level.

Third, I would strengthen citation verification. The model-assisted judge currently evaluates from the saved answer, citation metadata, and benchmark expectations; it does not refetch every source. A stronger citation auditor would perform claim-level source checking by retrieving cited passages.

Finally, I would explore a small architecture search over genome variants rather than the manually constrained versions here. The project already has the right representation: genomes, validation, traces, evaluation, acceptance criteria, and rollback. The next step would be to let the stem agent propose candidates automatically from evaluator feedback.

7 Conclusion

The project demonstrates a constrained version of a stem agent. The agent does not become universal, and it does not rewrite arbitrary code. Instead, it specializes by evolving an auditable genome for technical deep research. The best-quality evolved genome, v5, improved the final score over the web-search baseline from 0.6738 to 0.8569, but the stronger result is behavioral: the final agent leaves inspectable traces for decomposition, source triage, evidence extraction, coverage checking, contradiction handling, and citation auditing, while the baseline only searches and summarizes.

The main lesson is that specialization is valuable, but only when paired with evaluation and constraints. The first evolved genome did not beat the baseline; the coverage-injected one improved quality but became expensive; the budget-tuned one saved tokens but lost reliability; the source-quality one improved quality but exposed a citation-contract failure; the citation-contract one fixed that failure and reduced cost, but revealed a trace parsing robustness issue. That path is the most important part of the result: the system exposes not just a final score, but the engineering process by which the agent became specific, observable, and easier to debug.

References

[1] Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models.* ICLR, 2023.

[2] Schick et al. *Toolformer: Language Models Can Teach Themselves to Use Tools.* NeurIPS, 2023.

[3] Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning.* NeurIPS, 2023.

[4] Madaan et al. *Self-Refine: Iterative Refinement with Self-Feedback.* NeurIPS, 2023.

[5] Hu, Lu, and Clune. *Automated Design of Agentic Systems.* ICLR, 2025.

[6] Singh et al. *Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG.* arXiv, 2025.

[7] Yehudai et al. *Survey on Evaluation of LLM-based Agents.* CoRR/OpenReview, 2025.

[8] Zheng et al. *DeepResearcher: Scaling Deep Research via Reinforcement Learning in Real-world Environments.* EMNLP, 2025.